

NaviglA

Chatbot spécialisé dans les transports d'Île-de-France

Rapport de projet

BUT 3 Informatique

ENCADRANTS

M. Faye · Mme Azzag

RÉALISÉ PAR

Selma · Yacine · Kenza · Delrone · Mounir

26 juin 2026



Sommaire

1. Introduction	2
2. Objectifs du projet	2
3. Démarche de réalisation	2
3.1 Analyse du besoin	2
3.2 Collecte et structuration des données	2
3.3 Mise en place du RAG	3
3.4 Mise en place de la recherche internet	3
3.5 Développement du backend	3
3.6 Développement du frontend	4
4. Architecture des données	4
4.1 Deux types de stockage	4
4.2 Le modèle de données relationnel	5
4.3 SQLAlchemy comme ORM	5
4.4 De SQLite à PostgreSQL	5
4.5 ChromaDB pour le stockage vectoriel	6
5. Choix techniques	6
5.1 React et Vite pour le frontend	6
5.2 Flask pour le backend	6
5.3 SQLite pour les données applicatives	6
5.4 SQLAlchemy pour l'accès aux données	6
5.5 bcrypt pour les mots de passe	7
5.6 LangChain et ChromaDB pour le RAG	7
5.7 Hugging Face pour les embeddings	7
5.8 OpenRouter pour le modèle de langage	7
5.9 Tavily pour la recherche web contrôlée	7
6. Déploiement	8
6.1 Stratégie de déploiement	8
6.2 Préparation du code pour la production	8
6.3 Mise en ligne	8
6.4 Tests réalisés et résultats	9
6.5 Limites du déploiement	9
7. Fonctionnalités principales	9
7.1 Authentification	9
7.2 Gestion des conversations	9
7.3 Envoi de messages	9
7.4 Affichage des sources	10
7.5 Interface utilisateur	10
7.6 Interaction vocale	10
7.7 Responsive mobile	10
8. Limites du projet	10
9. Conclusion	10

1. Introduction

NaviglA est une application web full stack qui permet d'interroger des informations sur les transports en commun, avec un focus sur l'Île-de-France et les opérateurs SNCF, Transilien, RATP et Île-de-France Mobilités.

Concrètement, l'application prend la forme d'un chatbot spécialisé. L'utilisateur pose une question en français, l'application recherche l'information pertinente, puis renvoie une réponse structurée en indiquant ses sources. Il peut aussi retrouver l'historique de ses conversations. L'idée n'est pas de proposer un assistant généraliste de plus, mais un outil ciblé sur un domaine précis, qui s'appuie d'abord sur des données locales et n'utilise le web que lorsque c'est nécessaire.

Techniquement, le projet repose sur une interface React, une API Flask, une base de données relationnelle, un système RAG construit avec LangChain et ChromaDB, et un modèle de langage interrogé via OpenRouter.

2. Objectifs du projet

Les objectifs principaux sont les suivants :

- proposer une interface simple pour poser des questions sur les transports ,
- exploiter des jeux de données publics au format JSON ,
- mettre en place une recherche documentaire locale avec embeddings ,
- conserver les conversations des utilisateurs ,
- permettre l'inscription, la connexion et la déconnexion ,
- afficher les sources utilisées pour renforcer la transparence des réponses ,
- prévoir un fallback web limité à des domaines fiables.

3. Démarche de réalisation

3.1 Analyse du besoin

Le besoin que nous avons identifié pour un chatbot spécialisé dans les transports est celui d'accéder aux informations générales sur les transports en commun en Île-de-France. L'objectif n'est pas de créer un assistant de voyage avec création d'itinéraires en temps réel, etc., mais plutôt, d'obtenir des informations générales telles que la propreté des gares, l'accessibilité pour les personnes à mobilité réduite, les services présents en gare ou encore les tarifs des différents abonnements proposés.

L'application vise à servir d'intermédiaire entre l'utilisateur et ces données. L'utilisateur formule une question simple, puis le système recherche les informations pertinentes avant de générer une réponse cohérente.

3.2 Collecte et structuration des données

Les données officielles proposées par les différents organismes de transport sont principalement proposées sous forme de fichiers JSON, CSV ou encore GTFS (General Transit Feed Specification). En effet, les fichiers GTFS permettent aux réseaux de transport de publier leurs données dans un format commun et sont composés de plusieurs fichiers CSV. Le problème avec ces fichiers est qu'ils sont liés entre eux et qu'il est donc nécessaire

de créer des jointures entre eux, ce qui n'est pas adapté pour notre RAG. Ainsi, l'utilisation de fichiers JSON est plus adaptée dans notre cas, avec des informations structurées mais faciles à récupérer.

Les données locales sont stockées dans le dossier [data/json/](#). Elles couvrent notamment :

- les titres et tarifs Ile-de-France Mobilités ,
- les horaires de gares SNCF(mais pas les horaires des transports en temps réel),
- la fréquentation des gares,
- les équipements d'accessibilité,
- les données de propreté en gare.

Ces données sont récupérées directement sur les sites officiels des différents organismes de transports en communs, dont les liens sont disponibles dans le fichier [data/liens](#).

Ces fichiers ne sont pas utilisés tels quels par le modèle de langage. Ils sont chargés, transformés en documents textuels, enrichis avec des métadonnées, puis indexés dans une base vectorielle.

3.3 Mise en place du RAG

Le Retrieval-Augmented Génération (RAG), est au cœur du projet. Son principe est de rechercher d'abord des documents pertinents, puis de fournir ces documents en tant que contexte au modèle de langage pour l'aider à répondre.

Dans NavigIA, le pipeline RAG est implémenté dans [backend/core/rag.py](#). Il réalise les opérations suivantes :

1. chargement des fichiers JSON,
2. transformation des lignes de données en documents Lang Chain,
3. ajout de métadonnées: la source, le domaine du document et un id,
4. génération d'embeddings avec Hugging Face,
5. stockage dans ChromaDB,
6. recherche vectorielle lors d'une question utilisateur,
7. transmission du contexte pertinent au modèle de langage.

Les étapes 2 et 3 consistent à récupérer chaque objet JSON et à le transformer en document Lang Chain qui contient le texte contenu dans les objets JSON et auquel on ajoute les métadonnées. Au total nous avons 1 246 documents.

Puis, l'étape 4 consiste à transformer ces documents en vecteurs lors de l'embedding. Cet embedding est réalisé avec le modèle Hugging Face **paraphrase-multilingual-MiniLM-L12-v2**. Ces vecteurs sont ensuite stockés dans la base de données vectorielle ChromaDB lors de l'étape 5. Transformer les documents en représentation numérique permet ensuite de rechercher plus facilement et de manière plus fiable les informations pertinentes pour répondre à la question de l'utilisateur. Concrètement, la question de l'utilisateur est également transformée en vecteur et stockée dans ChromaDB. Ainsi, nous procédons à des comparaisons des distances euclidiennes entre les vecteurs questions et les vecteurs de nos données. Finalement, on récupère les vecteurs de

données les plus proches de ceux des questions et cela nous donne le contexte de réponse à transmettre au modèle de langage à l'étape 7.

Ce choix permet de réduire les réponses inventées, car le modèle reçoit un contexte limité et spécifique.

Nous utilisons le LLM, **google/gemma-4-31b-it:free** appelé via OpenRouter. Il reçoit d'abord un contexte extrait des documents structurés et officiels ainsi que la question de l'utilisateur. Puis, la réponse est guidée par des passages précis, associés à des métadonnées de source, et par un prompt qui permet de lui passer les mêmes contraintes à chaque fois. Dans ces contraintes on retrouve: répondre uniquement depuis le contexte, répondre en français, éviter les réponses avec des horaires précis et préciser dans la réponse si celle-ci est formulée à partir d'une recherche web (recherche internet détaillée dans la partie suivante). Cela améliore la fiabilité et permet d'indiquer clairement l'origine de l'information.

Concernant le modèle LLM choisi, nous avons réalisé ce choix pour des raisons pratiques: gratuité, capacité à suivre des instructions, réponses en français et capacité à utiliser un contexte RAG.

3.4 Mise en place de la recherche internet

Nous avons mis en place un fallback vers la recherche web afin de compléter le fonctionnement du RAG lorsque nos documents en locale ne contiennent pas les informations nécessaires à la question. Le chatbot interroge d'abord ChromaDB pour récupérer des documents issus des fichiers JSON. Si aucun contexte n'est trouvé (ou que ceux trouvés sont considérés comme trop éloignés), le backend lance une recherche web via **Tavily**. Les résultats sont ensuite filtrés pour ne conserver que des sources fiables comme SNCF, Transilien, RATP ou Île-de-France Mobilités. Le contexte web obtenu est transmis au LLM, qui génère une réponse en indiquant qu'elle s'appuie sur une source web.

3.5 Développement du backend

Le backend est développé avec Flask. Il expose les routes nécessaires à l'authentification et à la discussion. Ses principales responsabilités sont :

- gérer les utilisateurs,
- sécuriser les sessions avec des tokens,
- stocker les conversations,
- rechercher le contexte documentaire,
- appeler le modèle de langage,
- renvoyer les réponses et les sources au frontend.

Il est organisé en couches :

- `controllers/` pour les routes HTTP ;
- `services/` pour la logique métier ;
- `repositories/` pour l'accès aux données ;
- `core/` pour le RAG, le LLM et la recherche web ;
- `models.py` et `database.py` respectivement, pour définir les tables et configurer la connexion à SQLite.

Cette organisation permet d'avoir un code plus lisible et facile à maintenir.

3.6 Développement du frontend

Le frontend est développé avec React et Vite. Il est composé de:

- une page d'inscription,
- une page de connexion,
- une interface de discussion,
- une barre latérale avec l'historique des conversations,
- un champ de saisie avec une option de recherche web,
- l'affichage des réponses,
- l'affichage des sources (RAG ou web).

L'interface est sobre et instinctive. Elle reprend les mêmes codes que les interfaces d'IA déjà existantes sur le marché. L'objectif est de proposer un service pratique et facile à prendre en main.

Le frontend communique avec le backend via des requêtes HTTP.

4. Architecture des données

4.1 Deux types de stockage

Le projet s'appuie sur deux bases de données distinctes, chacune adaptée à un usage différent :

- une base relationnelle (SQLite en développement et PostgreSQL en production) pour les données applicatives : comptes utilisateurs, tokens d'authentification, sessions de conversation et messages,
- une base vectorielle (ChromaDB) pour stocker les embeddings des documents de transport, utilisés par le RAG pour la recherche sémantique.

Ce choix de séparer les deux types de stockage est justifié par la nature des données. Les données utilisateurs sont structurées et relationnelles (un utilisateur possède plusieurs sessions, une session contient plusieurs messages), ce qui correspond parfaitement à une base SQL. Les documents du RAG, en revanche, sont représentés sous forme de vecteurs numériques de haute dimension, et nécessitent une recherche par similarité qui n'est pas adaptée au SQL classique.

4.2 Le modèle de données relationnel

Le schéma relationnel est défini dans `backend/models.py` avec SQLAlchemy. Il comporte quatre tables :

Table	Rôle	Champs principaux
users	Comptes utilisateurs	id, username, email, password_hash, created_at
auth_tokens	Tokens de session	id, token, user_id
conversation_sessions	Conversations	id, title, created_at, updated_at, user_id

Table	Rôle	Champs principaux
messages	Messages échangés	id, role, content, source, created_at, session_id

Les relations entre ces tables sont les suivantes :

- un utilisateur possède plusieurs sessions et plusieurs tokens (relation one-to-many) ;
- une session contient plusieurs messages, ordonnés chronologiquement ;
- supprimer un utilisateur supprime en cascade ses sessions, ses messages et ses tokens.

Dans la table `messages`, le champ `role` distingue les messages de l'utilisateur (`user`) de ceux du chatbot (`assistant`). Le champ `source` indique l'origine de la réponse : `rag` pour les données locales, `web` pour une recherche internet, et `null` pour les messages utilisateur. Enfin, le champ `password_hash` stocke le mot de passe haché avec `bcrypt`, jamais en clair, conformément aux bonnes pratiques.

4.3 SQLAlchemy comme ORM

Nous interagissons avec la base via SQLAlchemy. Cet ORM nous permet de manipuler les données avec des objets Python plutôt qu'avec des requêtes SQL écrites à la main. Par exemple, pour récupérer toutes les sessions d'un utilisateur :

```
db.query(ConversationSession).filter_by(user_id=user.id).all()
```

Comme les modèles sont décrits en Python, cet outil nous a aussi beaucoup aidés au moment de migrer de SQLite vers PostgreSQL pour le déploiement.

4.4 De SQLite à PostgreSQL

En développement, la base est un simple fichier local `chatbot.db` au format SQLite. C'est pratique pour démarrer vite : SQLite est intégré à Python et ne demande aucun serveur à installer. En revanche, est limité pour la production: le fichier est stocké localement sur le serveur, les données disparaissent à chaque redémarrage si le disque est éphémère, et les accès simultanés de plusieurs utilisateurs sont mal gérés.

Pour la production, nous sommes donc passés à PostgreSQL, hébergé chez Neon, un service de base de données serverless. Grâce à SQLAlchemy, la migration s'est limitée à changer la chaîne de connexion dans les variables d'environnement sans modifier les modèles ni les requêtes. Cette migration a réglé deux problèmes d'un coup: la persistance, puisque les comptes et les conversations sont conservées même après un redémarrage, et les accès concurrents, PostgreSQL gérant nativement plusieurs connexions en parallèle.

Un problème spécifique à Neon a tout de même dû être résolu: la base se met en veille après une période d'inactivité, ce qui provoquait des erreurs « *SSL connection closed* ». Nous avons activé l'option `pool_pre_ping` de SQLAlchemy dans `backend/database.py`, qui teste la connexion avant chaque requête et la rouvre si besoin.

4.5 ChromaDB pour le stockage vectoriel

ChromaDB est notre base vectorielle pour le RAG. Elle stocke les embeddings générés à partir des documents de transport et permet de rechercher par similarité sémantique.

L'index est persisté localement dans le dossier `chroma_db/`. En production, il est reconstruit au démarrage du serveur à partir des fichiers JSON source, soit 11 246 documents.

5. Choix techniques

5.1 React et Vite pour le frontend

Nous avons retenu React pour le frontend parce qu'il pousse à découper l'interface en composants réutilisables : Sidebar, ChatWindow, MessageInput, MessageBubble, etc. Vite l'accompagne pour sa simplicité, son démarrage rapide et sa bonne intégration avec React. Nous avons écarté Angular : il impose davantage de structure et aurait été plus difficile à prendre en main, d'autant que certains membres de l'équipe ne l'avaient jamais utilisé.

5.2 Flask pour le backend

Pour le backend nous avons choisi Flask car c'est un framework python léger et adapté à une API simple. Il permet de structurer rapidement les routes.

Ce choix est cohérent avec la taille du projet : l'application a besoin d'un backend clair, mais pas d'un framework trop lourd.

5.3 SQLite pour les données applicatives

Pour les données applicatives (utilisateurs, tokens, sessions, messages) nous avons d'abord utilisé SQLite, qui permet de démarrer sans installer ni configurer de serveur. Pour la mise en production, nous avons ensuite migré vers PostgreSQL afin d'obtenir un stockage persistant et de supporter plusieurs utilisateurs en parallèle. Cette migration et l'architecture des données sont détaillées en section 4.

5.4 SQLAlchemy pour l'accès aux données

SQLAlchemy nous sert d'ORM : il nous permet de décrire les tables et les relations avec des modèles Python plutôt qu'avec du SQL brut, et c'est lui qui a rendu la migration vers PostgreSQL aussi simple.

5.5 bcrypt pour les mots de passe

La sécurité compte d'autant plus que l'application manipule des données utilisateurs. Plutôt que de stocker les mots de passe en clair, nous les hachons avec bcrypt avant de les enregistrer, comme le recommandent les bonnes pratiques.

5.6 LangChain et ChromaDB pour le RAG

Pour le stockage des embeddings nous avons choisi ChromaDB. En effet, nous avons besoin d'une base vectorielle pour stocker les représentations numériques des documents et de rechercher les documents utiles à la question de l'utilisateur. Ce qui n'est pas adapté à une base de données SQL.

5.7 Hugging Face pour les embeddings

Les embeddings sont générés par le modèle `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`, adapté à un contexte multilingue et donc aux questions en français. Il rapproche des formulations différentes mais

de sens proche : une question sur un « abonnement mensuel Navigo » peut ainsi retrouver un document qui parle de « Forfait Navigo Mois ».

5.8 OpenRouter pour le modèle de langage

Pour accéder au modèle de langage, nous passons par OpenRouter, une plateforme qui donne accès à différents LLM via une seule API. En pratique, les modèles gratuits sont soumis à des limites d'usage (rate limit, erreur 429) et peuvent être temporairement saturés ou indisponibles. Pour éviter toute coupure de service, nous avons mis en place une cascade de cinq modèles, essayés dans cet ordre :

- `openai/gpt-oss-120b:free`
- `google/gemma-4-31b-it:free`
- `liquid/lfm-2.5-1.2b-instruct:free`
- `nvidia/nemotron-3-super-120b-a12b:free`
- `qwen/qwen3-next-80b-a3b-instruct:free`

Le backend tente le premier modèle et bascule automatiquement sur le suivant en cas d'échec. Si tous échouent, un message d'attente est renvoyé à l'utilisateur.

Aucun de ces modèles n'est à proprement parler un modèle « français » : ce sont des modèles multilingues capables de produire des réponses de bonne qualité en français. Pour garantir une réponse en français quel que soit le modèle finalement utilisé, le prompt système impose explicitement la langue, ainsi que plusieurs contraintes déjà présentées plus tôt dans ce rapport.

5.9 Tavily pour la recherche web contrôlée

Pour les cas où l'information n'est pas présente dans notre jeu de données, nous avons ajouté un repli web via Tavily. Il intervient quand le RAG ne trouve pas de contexte suffisant, ou quand l'utilisateur active lui-même la recherche web. Les résultats sont filtrés pour ne conserver que des sources que nous jugeons fiables :

- `sncf.com`
- `transilien.com`
- `iledefrance-mobilites.fr`
- `ratp.fr`
- `service-public.fr`

Ce filtre nous évite de remonter des sources non officielles ou peu fiables.

6. Déploiement

6.1 Stratégie de déploiement

L'application a été mise en ligne en répartissant ses trois composants sur des plateformes d'hébergement gratuites, chacune adaptée à son rôle.

Le frontend (React + Vite) est hébergé sur Vercel, sous forme de fichiers statiques servis par un CDN (Content Delivery Network). Vercel se déploie automatiquement à chaque mise à jour du dépôt GitHub.

Le backend, qui regroupe l'API Flask et le système RAG, tourne sur Hugging Face Spaces, dans un conteneur Docker et derrière le serveur de production Gunicorn.

La base de données, enfin, est un PostgreSQL hébergé chez Neon : une base serverless, persistante et indépendante de l'hébergement du backend.

Le navigateur dialogue avec le frontend, qui appelle l'API du backend en HTTPS. Le backend interroge à son tour la base PostgreSQL, sa base vectorielle ChromaDB et le modèle de langage via OpenRouter.

6.2 Préparation du code pour la production

Passer du fonctionnement en local à la production a demandé plusieurs adaptations.

Nous avons d'abord ajouté Gunicorn et un point d'entrée `wsgi.py` : le serveur intégré de Flask ne sert qu'au développement et n'est pas fait pour un usage réel. Côté frontend, l'adresse de l'API a été sortie du code dans une variable d'environnement (`VITE_API_URL`), alors qu'un simple proxy suffisait en local.

Nous avons également configuré les origines autorisées à appeler l'API (CORS) via une variable d'environnement, et ajouté un fichier `vercel.json` pour rediriger toutes les routes vers `index.html`.

Enfin, les clés sensibles (OpenRouter, Tavily, accès à la base) sont gérées uniquement par des variables d'environnement et des secrets, et ne figurent jamais dans le code source.

6.3 Mise en ligne

Le backend est construit à partir d'un `Dockerfile`, qui récupère le code, installe les dépendances et lance Gunicorn. Au premier démarrage, l'index du RAG est reconstruit à partir des 11 246 documents collectés. Le frontend est déployé en faisant pointer `VITE_API_URL` vers l'adresse publique du backend, et la base Neon est reliée au backend par une chaîne de connexion conservée comme secret.

En production, l'application est accessible à l'adresse <https://transports-chatbot.vercel.app> et l'API à l'adresse <https://delrone-navigia-backend.hf.space>.

6.4 Tests réalisés et résultats

Une fois l'application en ligne, nous avons mené des tests manuels de bout en bout. Plusieurs fonctionnalités ont été validées: l'inscription et la connexion, la création, la sélection, le renommage et la suppression de conversations, l'envoi de questions et la génération de réponses à partir des données locales, le repli automatique sur la recherche web limitée aux sources officielles lorsque le RAG ne trouve rien, la persistance des données, un compte créé restant disponible après un redémarrage du serveur, ce qui valide le couple PostgreSQL/Neon, et la communication frontend-backend en HTTPS, sans erreur CORS.

Plusieurs problèmes ont dû être résolus en chemin. Le premier hébergeur envisagé, Render, s'est révélé inutilisable : son offre gratuite (512 Mo de RAM) ne suffisait pas pour PyTorch et le modèle d'embeddings, ce qui nous a conduits à migrer vers Hugging Face Spaces (environ 16 Go). Le fichier de données a ensuite été refusé par Hugging Face, dont le dépôt limite chaque fichier à 10 Mo ; le `Dockerfile` récupère donc désormais le code et les données

depuis GitHub au moment de construire l'image. L'index du RAG n'était pas non plus construit en production, son initialisation se trouvant dans un bloc de code que Gunicorn n'exécutait pas : nous l'avons déplacée dans `wsgi.py`.

D'autres ajustements ont suivi. Les données étaient perdues à chaque redémarrage parce que SQLite était stocké sur un disque éphémère, d'où le passage à PostgreSQL (Neon). Une erreur « SSL connection closed » apparaissait lorsque Neon mettait la base en veille : nous avons activé l'option `pool_pre_ping` de SQLAlchemy. Enfin, des erreurs 404 survenaient sur certaines routes (par exemple un accès direct à `/login`), corrigées par l'ajout du fichier `vercel.json`.

6.5 Limites du déploiement

Le recours à des offres gratuites impose quelques contraintes. Sur Hugging Face, le Space se met en veille après environ 48 heures d'inactivité et à son réveil, il faut compter deux à trois minutes de démarrage à froid, le temps de re-télécharger le modèle et de reconstruire l'index du RAG. La base Neon gratuite est par ailleurs limitée à 0,5 Go de stockage, ce qui reste largement suffisant pour le volume de ce projet.

7. Fonctionnalités principales

7.1 Authentification

L'utilisateur peut créer un compte et se connecter. Une fois connecté, un token est généré et stocké côté client pour authentifier ses requêtes suivantes.

7.2 Gestion des conversations

Chaque utilisateur dispose de ses propres sessions de discussion. Il peut créer une conversation, en consulter une existante et la supprimer ; la session est par ailleurs renommée automatiquement à partir du premier message.

7.3 Envoi de messages

Quand un utilisateur envoie un message, le backend :

- enregistre le message utilisateur ;
- recherche un contexte avec le RAG ;
- fait appel à Tavily si nécessaire ;
- construit les messages destinés au LLM ;
- appelle OpenRouter ;
- enregistre la réponse de l'assistant ;
- renvoie la réponse et les sources au frontend.

7.4 Affichage des sources

Le frontend indique si la réponse vient des données locales ou d'une recherche web vérifiée. Lorsqu'il y a des sources web, les liens sont présentés à l'utilisateur.

7.5 Interface utilisateur

L'interface comprend des pages d'authentification simples, un espace de conversation, une barre latérale pour l'historique, des suggestions de questions, un champ de saisie clair et un affichage des échanges sous forme de bulles de messages.

7.6 Interaction vocale

L'application propose une saisie vocale grâce à l'API Web Speech Recognition du navigateur. En cliquant sur le bouton microphone, l'utilisateur peut dicter sa question en français : la transcription s'insère dans le champ de saisie, puis part comme un message texte classique. La fonctionnalité est disponible sur les navigateurs compatibles (Chrome, Edge).

7.7 Responsive mobile

L'interface s'adapte aux écrans mobiles : la barre latérale devient un menu coulissant, les bulles de messages prennent toute la largeur et les tableaux défilent horizontalement.

8. Limites du projet

Le projet fonctionne comme un prototype abouti, mais plusieurs limites méritent d'être signalées :

- les informations en temps réel ne sont pas garanties ;
- la qualité du RAG dépend directement de celle des données JSON ;
- l'index ChromaDB doit être régénéré si les données changent beaucoup ;
- les hébergements gratuits imposent un démarrage à froid après une longue période d'inactivité.

9. Conclusion

Ce projet nous a permis de mettre en pratique une chaîne full stack complète, de l'interface utilisateur jusqu'au déploiement en production.

L'approche RAG s'est révélée efficace pour fournir des réponses fiables, appuyées sur des données officielles. Plusieurs pistes d'amélioration restent ouvertes : ajouter des données en temps réel via les API SNCF/RATP, intégrer un calcul d'itinéraire, ou encore mettre en place des tests automatisés.